

SQL Showdown: 20 Key Differences Every Data Professional Must Know



Pooja Pawar



1. Primary Key vs. Foreign Key

Aspect	Primary Key	Foreign Key
Purpose	Uniquely identifies a record in a table.	Links tables by referencing a primary key.
Duplicates	Not allowed.	Allowed.
Nulls	Not allowed.	Allowed.
Attachment	Attached to the table being defined.	Attached to the referencing table.
Index	Creates a unique index by default.	Does not create an index by default.
Usage	Used for identifying records.	Used for maintaining referential integrity.

2. DELETE vs. TRUNCATE

Aspect	DELETE	TRUNCATE
Purpose	Removes specific rows.	Removes all rows from a table.
Rollback	Can be rolled back.	Cannot be rolled back.
Triggers	Activates triggers if present.	Does not activate triggers.
Speed	Slower as it logs individual deletions.	Faster as it doesn't log individual rows.
Table Structure	Preserves table structure and indexes.	Resets table structure and indexes.
Condition	Supports WHERE clause for selective removal.	Does not support WHERE clause.

3. WHERE vs. HAVING

Aspect	WHERE	HAVING
Filter Type	Filters rows before grouping.	Filters groups after aggregation.
Used With	SELECT, UPDATE, DELETE.	Only SELECT.
Aggregate Functions	Not supported.	Supported.
Execution Order	Applied before GROUP BY.	Applied after GROUP BY.
Filtering	Filters individual rows.	Filters aggregated rows.
Use Case	For row-level conditions.	For group-level conditions.

4. RANK vs. DENSE_RANK

Aspect	RANK	DENSE_RANK
Ranking Gaps	Skips ranks when there are ties.	No gaps in ranking.
Ideal Use	When gaps in ranking are acceptable.	When continuous ranking is required.
Syntax	RANK() OVER (...).	DENSE_RANK() OVER (...).
Performance	Slightly slower due to gaps in ranks.	Faster for consecutive ranking.
Result Example	1, 2, 2, 4.	1, 2, 2, 3.

5. VIEW vs. MATERIALIZED VIEW

Aspect	VIEW	MATERIALIZED VIEW
Existence	Logical, does not store data.	Physical, stores data.
Data Source	Fetches data from base tables when queried.	Fetches data from stored results.
Auto-Refresh	Cannot auto-refresh.	Can be auto-refreshed.
DML Operations	Limited to non-complex views.	Supported for most operations.
Storage	Does not occupy storage space.	Occupies storage space.
Use Case	For frequently updated data.	For pre-computed data.

6. Stored Procedure vs. Function

Aspect	Stored Procedure	Function
Return Type	May or may not return values.	Must return a value.
Purpose	For tasks and operations.	Primarily for calculations.
Call from SQL	Cannot be called in SELECT.	Can be called in SELECT.
Arguments	Can accept multiple arguments.	Accepts arguments and returns one value.
Compilation	Pre-compiled.	Compiled at runtime.

7. OLTP vs. OLAP

Aspect	OLTP	OLAP
Purpose	Transactional processing.	Analytical processing.
Normalization	Highly normalized.	Often denormalized.
Query Type	Simple and frequent queries.	Complex and less frequent queries.
Typical Use	Banking, e-commerce.	Data warehousing, reporting.
Focus	Speed of operations.	Query performance.

8. UNION vs. UNION ALL

Aspect	UNION	UNION ALL
Duplicates	Removes duplicates.	Retains duplicates.
Performance	Slower due to duplicate removal.	Faster as no duplicate removal is done.
Use Case	When unique results are needed.	When all results are required.
Syntax	SELECT ... UNION SELECT	SELECT ... UNION ALL SELECT

9. Star Schema vs. Snowflake Schema

Aspect	Star Schema	Snowflake Schema
Complexity	Simpler structure.	More complex structure.
Joins	Fewer joins required.	Multiple joins required.
Normalization	Less normalized.	Highly normalized.
Performance	Faster queries.	Slower due to joins.
Hierarchy	No hierarchy between dimension tables.	Hierarchies exist between dimension tables.

10. Temporary Tables vs. CTE

Aspect	Temporary Tables	CTE (Common Table Expressions)
Scope	Exists for the session.	Exists only for the query.
Reusability	Can be reused in multiple queries.	Not reusable outside the query.
Storage	Stored in tempdb.	Not stored; exists in memory only.
Performance	Slower due to storage requirements.	Faster as it is memory-based.
Syntax	CREATE TABLE #TempTable.	WITH CTE AS (...).

11. LEAD vs. LAG

Aspect	LEAD	LAG
Purpose	Retrieves data from the next row.	Retrieves data from the previous row.
Use Case	Future trends or forecasts.	Past trends or comparisons.
Offset	Default is 1 (next row).	Default is 1 (previous row).
Syntax	LEAD(column, offset, default) OVER (...).	LAG(column, offset, default) OVER (...)
Calculation Type	Useful for forward-looking analysis.	Useful for historical analysis.

12. Sub-Query vs. Correlated Sub-Query

Aspect	Sub-Query	Correlated Sub-Query
Execution	Executed once for the parent query.	Executed for each row of the parent query.
Dependency	Independent of the parent query.	Depends on values from the parent query.
Complexity	Simpler to implement.	More complex due to row-wise execution.
Performance	Faster due to single execution.	Slower due to multiple executions.
Use Case	Static conditions.	Dynamic row-based conditions.

13. CHAR vs. VARCHAR

Aspect	CHAR	VARCHAR
Storage	Fixed-length.	Variable-length.
Padding	Pads extra spaces for shorter strings.	Does not pad extra spaces.
Performance	Faster for fixed-length data.	Slower due to dynamic length handling.
Use Case	Consistent-length fields (e.g., codes).	Varying-length fields (e.g., names).
Memory Usage	Consumes more space for shorter strings.	Optimized for storage.

14. DROP vs. TRUNCATE

Aspect	DROP	TRUNCATE
Effect	Deletes table structure and data.	Deletes all rows but keeps structure.
Rollback	Cannot be rolled back.	Cannot be rolled back.
Triggers	Does not activate triggers.	Does not activate triggers.
Use Case	Removing a table completely.	Clearing data while retaining structure.
Performance	Slightly slower.	Faster for clearing data.

15. COALESCE vs. ISNULL

Aspect	COALESCE	ISNULL
Purpose	Returns the first non-NULL value.	Replaces NULL with a specified value.
Arguments	Supports multiple arguments.	Supports only two arguments.
Portability	Works across multiple RDBMS.	Specific to SQL Server.
Usage	More versatile for complex conditions.	Simple replacement for NULLs.
Performance	Slightly slower due to flexibility.	Faster for simple cases.

16.INNER JOIN vs. OUTER JOIN

Aspect	INNER JOIN	OUTER JOIN
Result	Returns only matching rows.	Returns all rows, with NULLs for non-matching rows.
Variants	No variants.	Includes LEFT, RIGHT, and FULL OUTER JOIN.
Use Case	When matches are mandatory.	When all data from one or both tables is required.
Performance	Faster due to fewer rows.	Slower for larger datasets.
NULL Handling	Does not show NULLs for unmatched rows.	Shows NULLs for unmatched rows.

17. Temporary Table vs. Table Variable

Aspect	Temporary Table	Table Variable
Scope	Exists for the session.	Exists within the batch or procedure.
Storage	Stored in tempdb.	Stored in memory.
Performance	Slower due to disk I/O.	Faster for smaller datasets.
Constraints	Supports indexes and constraints.	Limited support for indexes and constraints.
Reusability	Can be reused across multiple queries.	Limited to a single batch or scope.

18.CROSS JOIN vs. FULL OUTER JOIN

Aspect	CROSS JOIN	FULL OUTER JOIN
Result	Produces the Cartesian product of rows.	Combines results of LEFT and RIGHT JOIN.
Matching Condition	No matching condition required.	Matches based on condition.
Row Count	Multiplies rows from both tables.	Combines rows from both tables.
Use Case	For pairing all rows.	For retrieving all data, matched or unmatched.

19.DISTINCT vs. GROUP BY

Aspect	DISTINCT	GROUP BY
Purpose	Removes duplicate rows.	Groups rows with aggregate functions.
Aggregation	Does not support aggregation.	Supports aggregation like SUM, COUNT.
Result	Shows unique rows.	Shows aggregated data.
Performance	Faster for small datasets.	Better for complex grouping operations.

20.Index vs. Partitioning

Aspect	Index	Partitioning
Purpose	Speeds up query performance.	Divides data into smaller, manageable segments.
Storage	Does not change data storage.	Physically organizes data.
Use Case	For frequently queried columns.	For large tables to optimize performance.
Performance	Improves query speed with quick lookups.	Balances load across partitions.